On the Applications of Partition Diagrams for Integer Partitioning

Rung-Bin Lin
Department of Computer Science and Engineering
Yuan Ze University
135 Yuan-Tung Road, Chung-Li, 320 Taiwan
csrlin@cs.yzu.edu.tw

Abstract

A partition diagram is a data structure for storing all the partitions of an integer n. In this paper, we first show how we extend a partition diagram to deal with integer partitioning with part restriction. We then show how to use a partition diagram to count the number of partitions in $O(n^2)$ time, count the number of occurrences for each integer in all partitions in $O(n^2)$ time, and generate a random partition in O(n) time.

1 Introduction

Partitioning a positive integer *n* is to divide the integer into its constituent parts which are all positive integers. All the constituent parts are said to form a partition of *n*. Algorithms for enumerating all the partitions of an integer or only the partitions with a restriction have long been invented [1-8]. A data structure called partition diagram for storing all the partitions of an integer is proposed in [9]. It takes $O(n^2)$ time and space to create a partition diagram. A partition diagram proposed in [9] is only for storing the partitions without any restriction on the set of allowable parts. In this paper we extend this data structure to handle the partitioning with a restriction on the set of allowable parts. In addition to this, we also show how we use a partition diagram to count the number of partitions in $O(n^2)$ time, count the number of occurrences for each integer in all partitions in $O(n^2)$ time, and generate a random partition in $O(n)$ time, respectively.

The rest of this paper is organized as follows. Section 2 briefly describes how to create a partition diagram. Section 3 presents our approach to forming a partition diagram with part restriction. Section 4 discusses how a partition diagram can be used to count the number of partitions, count the number of occurrences for each integer in all partitions, and randomly generate a partition. The last section draws some conclusions.

2 Partition Diagrams

Let $\langle y_1, y_2, ..., y_k \rangle$ be a partition of a positive integer n where $\sum_{i=1}^{k} y_i = n$ for $y_i \in I, y_i > 0, i = 1,...,k \le n$ and y_i is called a part of partition. We first assume that any positive integer not greater than *n* can serve as a part of a partition. The parts of a partition are not necessarily distinct, nor do they have a fixed order. However, in order to ease the representation, we assume $y_1 \le y_2 \le ... \le y_k$. For example, the partitions of 6 are ⟨1,1,1,1,1,1⟩, ⟨1,1,1,1,2⟩, ⟨1,1,1,3⟩, ⟨1,1,2,2⟩, ⟨1,1,4⟩, ⟨1,2,3⟩, ⟨1,5⟩, ⟨2,2,2⟩, ⟨2,4⟩, ⟨3,3⟩, and ⟨6⟩. Note that the value of *k* can vary from one partition to another.

The partition diagram of an integer *n* is a directed acyclic graph. Figure 1 shows a partition diagram of integer 6. A node in a partition diagram is denoted by (*y*,*Y*), where *y* is a part of a partition and *Y* is a number remained to be divided into parts that are not smaller than *y*. A node (*y*,*Y*) that has no predecessor is called an *anchored node* in a partition diagram. A node (*y*,*Y*) with *Y*=0 is called a *terminal node*. A terminal node does not have a successor. A node (*y*,*Y*) with $Y > 0$ and $y \le Y$ is called an *internal node*. A node (*x*,*X*) is a *successor* of a node (*y*,*Y*) if and only if $x + X = Y$ and $x \ge y$. A node (*y*,*Y*) with $Y \le n$ is called a *stem node* if *y* is the smallest among the allowable parts. For example, (1,6) is an anchored node, an internal node, and also a stem node, whereas (5,0) is a terminal node and (1,4) is a stem node. Given any two successors (*x*,*X*) and (x', X') of a node (*y*,*Y*), we have $x + X = x' + X' = Y$. All the successors of a stem node are implicitly stored in an array with increasing part values. An internal node (*y*,*Y*) which is not a stem node has an edge pointing to a node (*x*,*X*) where $x + X = Y$, $x \ge y$, and *x* is the smallest among all the nodes stored in the same array as (*x*,*X*). This edge implies that any other node (x', X') stored in the same array as (*x*,*X*) with $x' > x$ will also be a successor of node (*y*,*Y*). For example, we have an edge pointing from (2,4) to (2,2), it means both (2,2) and (4,0) are the successors of (2,4).

Given a partition diagram of an integer n, a path starting from an anchored node and stopping at a terminal node defines a unique partition of n. The partition is formed by all the parts excluding the part associated with the anchored node encountered during path traversal. For example, path (1,6) (1,5) (1,4) (2,2) (2,0) defines partition ⟨1,1,2,2⟩. Although the partition diagram shown in Figure 1 is created for integer 6, it consists of all the partition diagrams of any integer smaller than 6. The partition diagram of any integer r smaller than n is anchored at node (1,r). For example, (1,5) is the anchored node of the partition diagram of 5.

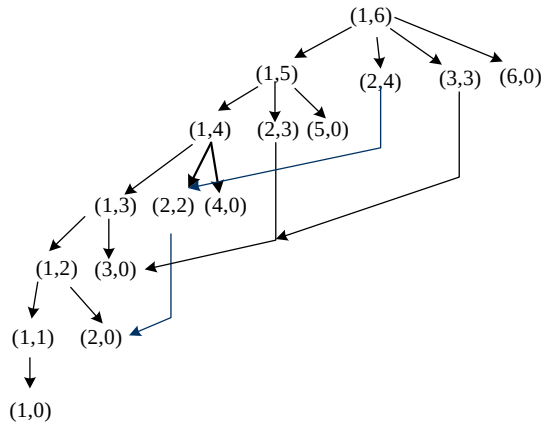

Figure 1. A partition diagram of 6.

3 Partition Diagram with Part Restriction

A partition diagram without part restriction can be created recursively for each of the integers starting from 1 to n [9]. In this section we show how a partition diagram can be used to deal with partitions whose parts can only be drawn from a set of allowable parts $S=\{s_1,s_2,\ldots,s_m\}$. Let s_1 be the smallest part. Now we would like to create a partition diagram for all positive integers not greater than n using only the parts in S. Before doing this, we first look at how a partition tree (or forest) can be created using only parts in S. We can obtain a partition tree from expanding the sharing of data structure in the corresponding partition diagram. The details of a partition tree are described in [9]. Figure 2 shows two partition trees for n=12 and n=11 with S={2,3,5}. These two trees form a forest and can be used to store all the partitions of any integer $r \le 12$. This is different from the tree created for the partitions of $n = 12$ without part restriction, i.e., only a single tree is created. The partition tree for integer n without part restriction consists of all the partition trees for positive integers smaller than n. Such a partition tree can be efficiently created using a top-down approach. However, creating a partition tree with part restriction can not be done efficiently with a top-down approach. The problem with a top-down approach is that we can not easily decide whether a child node such as (3,9) of (2,12) should be created or not. It is created only if 9 can be further partitioned into the parts all belonging to S. This information is not readily available with a top-down approach. On the contrary, this can be done with a bottom-up approach. Before we build a partition tree of integer r, we have already built the partition trees of all positive integers smaller than r. Thus, given any node (y,r-y), $y \in S$ we can easily decide whether node (y,r-y) should be created or not. What we do is to check whether there exists any node (a,A) in an already established tree such that the following condition is satisfied.

$$a + A = r - y \quad \text{and} \quad a \ge y . \qquad (1)$$

If such a node exists, node (y,r-y) should be created. For example, node (3,9) = (3,12-3) should be created because we can find at least a node (a,A)=(3,6) satisfying (1). That is, there exists a (2,9)'s child node whose part is equal to or larger than 3. This also implies that any sub-tree rooted at a child node of (3,9) will be also a sub-tree rooted at a child node of (2,9). Therefore, sharing of data structure occurs naturally in such a situation. An example contrary to this is that node (3,7)=(3,10-3) should not be created because there does not exist any (2,7)'s child node whose part is equal to or greater than 3. Note that all child nodes (a,A)'s of node (y,r-y) are stored in ascending order of their parts in an array which are readily accessible. Also note that there may exist more than one partition tree for the partitions of all positive integers not greater than n.

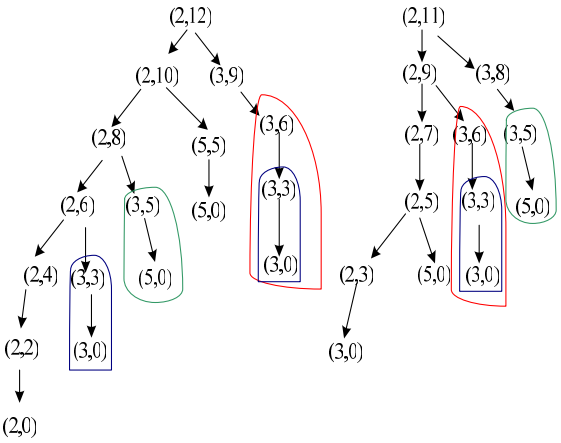

Figure 2. Partition trees of n=11 and n=12 with part restriction.

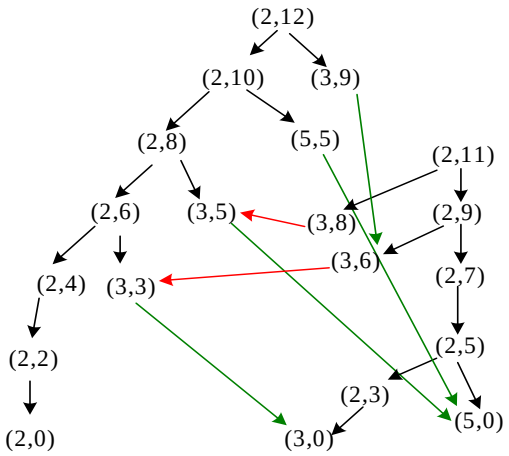

Figure 3. A partition diagram of n=12 with part restriction.

From the above discussion, we realize that data structure sharing is also highly probable in a partition diagram with part restriction. An example of a partition diagram for n=12 is given in Figure 3. Note that a node (y,Y) is a *stem node* if $y = s_1$. For example, (2,0), (2,2), (2,3), (2,4), …, (2,12) are stem nodes in the partition diagram for n=12. A stem node (s_1,r) may not exist if r can not be partitioned into the parts all belonging to S.

Now, let us look at the algorithm shown in Figure 4 for creating a partition diagram with part restriction. We iterate index i from s_1 through n to build a partition diagram for all integers not greater than n. In each iteration i=r, we have to count the number of successors that can be legally created for a predecessor (s_1,r). When performing this task, for each yet-to-be created successor $(y,r\text{-}y)$, $y \geq s_1$, we must find out the first $(y,r\text{-}y)$'s successor (a,A) and the total number of successors that satisfy (1). Note that we here refer two different levels of successors, a yet-to-be created successor $(y,r\text{-}y)$ of (s_1,r) and an already created successor (a,A) of $(y,r\text{-}y)$. A successor $(y,r\text{-}y)$, $y \in S$, of (s_1,r) can only be created if there exists at least a node (a,A) in the partition diagram such that (1) holds. Therefore, successor (2,8) can be created for (2,10), so can successor (5,5). However, (3,7) can not be created. When a stem node $(y,r\text{-}y)$, y=s_1, is created, a pointer pointing from the stem node $(s_1,r\text{-}s_1)$ to $(s_1,r\text{-}2s_1)$ should be established. If a non-stem node $(y,r\text{-}y)$, $y > s_1$ is created, a pointer pointing from node $(y,r\text{-}y)$ to the first shared successor of $(s_1,r\text{-}y)$ should be created. Note that sharing of data structure can only occur for a non-stem and non-terminal node. In the above example, we know that node $(y,r\text{-}y)$=(2,8) should be created as a successor of (s_1,r)=(2,10). In the mean time, the number of successors of (2,8), which is two, must also be registered along with the node. Similarly, $(y,r\text{-}y)$=(5,5) should be created and a pointer pointing to node (5,0), which is the second successor of (2,5) and the first successor of (5,5), should also be established. We also have to calculate the number of (2,5)'s successors that can be shared with (5,5), which is one in this example. The above tasks are fulfilled by the code from lines 2 to 8 in Figure 4.

Finally, we have to create an anchored node for the node that is created for each i=r and has not yet been referenced in the main for-loop. For the example shown in Figure 3, the anchored nodes (2,11) and (2,12) are created. If the part set S consists of 1, the partition diagram for any integer n will have only one anchored node. Clearly, there does not exist any directed edge between two anchored nodes. Given a set of parts S=$\{s_1,s_2,\ldots,s_m\}$ with s_1 being the smallest part, the number of anchored nodes is ranged from 1 to s_1. The reason is that any of the nodes (s_1,n), $(s_1,n\text{-}1)$, …, $(s_1,n\text{-}s_1\text{+}1)$ does not have a predecessor so that they must be anchored nodes if they exist.

The number of nodes used in the partition diagram of integer n depends on n and S, but it is bounded by $O(n^2)$. For a smaller set of S, the number of nodes will be much smaller than $O(n^2)$. Time complexity is the same as space complexity.

```
    void integer_partition_diagram_pr (int n){
(1)   for (i=s1; i<=n; i++){
(2)  num_of_s=
      count_number_of_successors(i);
(3)    if (num_of_s > 0) {
(4)      create an array of num_of_s nodes;
(5)      for (k=0; k<num_of_s; k++){
(6)        enter k-th node's data;
             // including its part, number of its
                successors, and the index to its first
                successor.
           } // end of for
(7)    else if (i is an allowed part)
(8)        create a node with part equal to i;
       } // end of for
(9)   create anchored nodes;
     }
```

Figure 4. Algorithm for creating a partition diagram with part restriction.

4 Applications of Partition Diagrams

As it is shown in [9], a partition diagram can be used to enumerate all the partitions embedded in it. In this section we show how it can be used to

count the number of partitions, count the number of occurrences of an integer in all partitions, and generate a random partition.

4.1 Counting Partitions

During creating a partition diagram, the number of terminal nodes (can be repeatedly counted) reachable from an internal node by different paths can be calculated and registered in the node. This can be done using a third field in the nodes as shown in Figure 5. Clearly, the number of terminal nodes reachable from a terminal node is 1, whereas the number of terminal nodes reachable from an internal node (a node with a successor) is the sum of all terminal nodes reachable from its successors. For example, the number of terminal nodes reachable from node (2,4) is 2, which is the sum of the number of terminal nodes reachable from (2,2) and (4,0). Once a partition diagram is created for integer n, we know the number of partitions of any integer smaller than or equal to n. All this can be done without increasing time and space complexity. Therefore, creating a partition diagram is an alternative algorithm with $O(n^2)$ complexity to the traditional approach that uses the basic identity $P_n(k) = P_{n-k}(k) + P_n(k-1)$ for calculating the number of partitions of an integer, where $P_n(k)$ is the number of n's partitions whose parts are not larger than k. The work in [8] also proposes an algorithm with $O(n^2)$ complexity to count the number of partitions of n. However, only partition diagram renders a data structure for storing all partitions of an integer.

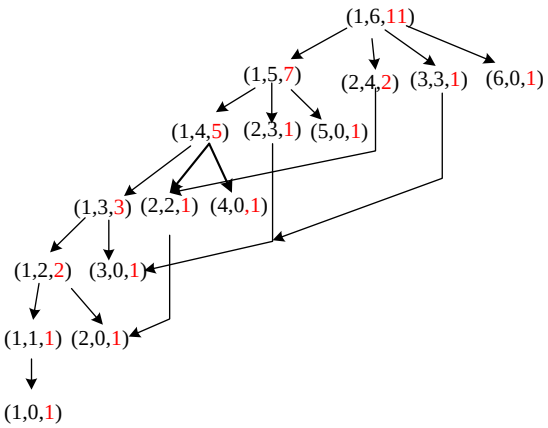

Figure 5. Counting the number of partitions

4.2 Counting Occurrences of an Integer

The number of occurrences of an integer in all partitions of an integer can be calculated if we also record the number of paths starting from the anchored node and terminated at a certain node as shown in the fourth field of each node in Figure 6.

For example, considering the node (4,0,1,2), the number of paths terminated at it is 2, one by way of (1,6,11,1), (2,4,2,1) and the other by way of (1,6,11,1)(1,5,7,1)(1,4,5,1). To record such information in each node, what we need is to make a breadth-first traversal of the diagram starting from the anchored node. Once we have the third and fourth fields, multiplying the third and fourth fields in a node we obtain the number of paths, each of which starts from the anchored node, traverses the underlying node, and terminates at a terminal node. This is equivalent to computing the number of partitions that contain the part specified in the underlying node. For example, the partitions containing the part associated with node (2,2,1,2) is $\langle 1,1,2,2\rangle$ and $\langle 2,2,2\rangle$. To compute the total number of occurrences of an integer in all partitions, we simply sum all the partitions containing that integer. For example, to find the number of occurrences of 2 we sum the number of partitions containing 2 at nodes (2,0,1,3), (2,2,1,2),(2,3,1,1), and (2,4,2,1), which is equal to 8. By the same way, we have 19 occurrences of 1, 4 occurrences of 3, 2 occurrences of 4, 1 occurrence of 5, and 1 occurrence of 6. All this can be done in $O(n^2)$ time. Note that the anchored node should not be considered in the counting of occurrences of 1.

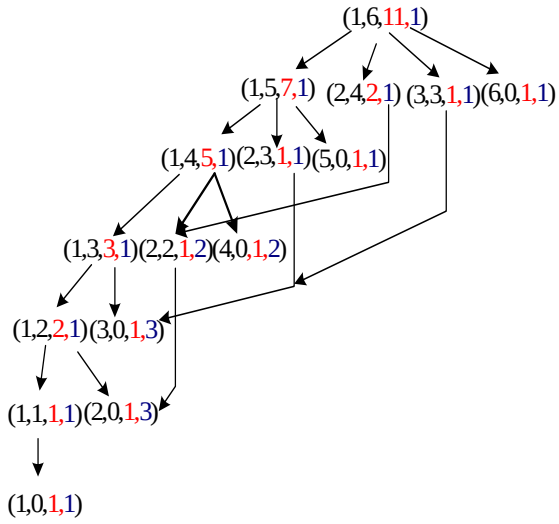

Figure 6. Counting the number of occurrences of an integer.

4.3 Generating a Random Partition

To generate a partition randomly we can simply use a partition diagram in Figure 5. Given a node (y,Y,p) in Figure 5, the third field p tells us the number of ways to divide Y into its constituent parts each not smaller than y. For example, node (2,4,2) tells us there are 2 ways to divide 4 into its

constituent parts each not less than 2, i.e., ⟨2,2⟩ and ⟨4⟩. Thus, we can start from the anchored node (1,*n*,*a*) and generate a random number *r* in [1,*a*]. We use the value *r* to decide which successor is traversed next. Supposed node (1,*n*,*a*) has *m* successors, (y_1, Y_1, p_1) , (y_2, Y_2, p_2) , …, (y_m, Y_m, p_m) , where $\sum_{i=1}^{m} p_i = a$. The *i*-th successor is selected if $\sum_{j=1}^{i-1} p_j < r \le \sum_{j=1}^{i} p_j$. The part specified in a successor is taken as one of the parts contained in a random partition. Let's say a successor (*y*,*Y*,*p*) is chosen where *y* will be the first part of a random partition and $Y = n - y$ is the number yet to be divided. The above procedure is repeated for this successor to find the second part, third part, …, until a terminal node is reached. For example, if a random number 9 is generated when we visit the anchored node, the successor (2,4,2) will be selected and 2 will be included in a random partition. At node (2,4,2) a random number in [1,2] should be generated. Supposed 2 is generated so that (4,0,1) is selected. Since a terminal node is encountered, this process stops and thus a random partition ⟨2,4⟩ is generated. Our experimental results show that this algorithm can correctly generate all the partitions each with equal probability. The following theorem shows formally that this algorithm has an O(*n*) time complexity.

Theorem 1: The time complexity to generate a random partition using a partition diagram is O(*n*).
Proof: The complexity consists of two components. The first component is the time to generate random numbers. Clearly, the algorithm at most generates *n* random numbers. The second component is the comparisons made to find a series of successors that form a path from the anchored node to a terminal node. Let the path be $(y_0, Y_0)(y_1, Y_1)(y_2, Y_2)...(y_k, Y_k)$, where $y_0 = 1$, $Y_0 = n$, and $Y_k = 0$. Here we ignore the third field in a node for simplicity. Then, we can show that the number of comparisons made to generate a random partition is either

$$C = \sum_{i=1}^{k-1} (y_i - y_{i-1} + 1) + \lfloor Y_{k-1}/2 \rfloor - y_{k-1} + 2 \quad \text{if } y_{k-1} \le \lfloor Y_{k-1}/2 \rfloor, \tag{2}$$

or

$$C = \sum_{i=1}^{k-1} (y_i - y_{i-1} + 1) + 1 \quad \text{if } Y_{k-1} \ge y_{k-1} > \lfloor Y_{k-1}/2 \rfloor. \tag{3}$$

In (2) and (3), $y_i - y_{i-1} + 1$ is the number of comparisons made for going from (y_{i-1}, Y_{i-1}) to (y_i, Y_i) for $i < k$. However, the number of comparisons made for going from (y_{k-1}, Y_{k-1}) to (y_k, Y_k) depends on whether $y_{k-1} \le \lfloor Y_{k-1}/2 \rfloor$. Since the number of successors of (y_{k-1}, Y_{k-1}) is equal to $\lfloor Y_{k-1}/2 \rfloor - y_{k-1} + 2$ if $y_{k-1} \le \lfloor Y_{k-1}/2 \rfloor$ and (y_k, Y_k) is the last successor of (y_{k-1}, Y_{k-1}), we need $\lfloor Y_{k-1}/2 \rfloor - y_{k-1} + 2$ comparisons to find whether (y_k, Y_k) should be included in the path. Thus, with $Y_{k-1} \le n - k + 1$ and $k \le n$ we can prove that

$$C \le k + Y_{k-1}/2 \le k + (n - k + 1)/2 \le n + 1/2 . \tag{4}$$

Since the number of comparisons must be an integral number, thus $C \le n$.
On the contrary, if $Y_{k-1} \ge y_{k-1} > \lfloor Y_{k-1}/2 \rfloor$, (y_k, Y_k) is the only successor of (y_{k-1}, Y_{k-1}) and thus we have (3). We can prove that

$$C \le k - 1 + y_{k-1} \le k - 1 + n - (k - 1) = n . \tag{5}$$

Since the number of comparisons and the number of random numbers needed to generate a random partition each are not greater than *n*, we complete the proof of this theorem. □

Although the above proof is provided for a partition diagram without part restriction, a similar one can be made to show that a partition diagram with part restriction can be employed to generate a random partition with O(*n*) time.
A simple inspection can be made to show (2) and (3) are correct for the partition diagram without part restriction. For example, suppose we have a random partition ⟨3,3⟩ that takes the path (1,6)(3,3)(3,0). Then, the number of comparisons made for going from (1,6) to (3,3) is 3 and it is 1 for going from (3,3) to (3,0). The total number of comparisons is 4 which can be obtained using (3). If a random partition ⟨1,1,1,3⟩ that takes the path (1,6)(1,5)(1,4)(1,3)(3,0), the number of comparisons made to generate such a partition is 5 which can be derived using (2).

5 Conclusions

In this paper we have proposed a method to create a partition diagram to store all partitions of an integer *n* with part restriction. In addition to this, we show how we use a partition diagram to count the number of partitions in O(n^2) time, count the number of occurrences of each integer in all partitions in O(n^2) time, and randomly generate a partition of an integer *n* in O(*n*) time. More applications of this data structure are yet to be

discovered. One may seek possibility of using a partition diagram to explain the occurrence of congruences in the number of partitions or to find new identities about the partitions of an integer [10].

6 References

[1] C. Tomasi, Two simple algorithms for the generation of partitions of an integer, Alta Frequenza, v51, n6 (1982) pp. 352-356.

[2] T. I. Fenner and G. Loizou, Analysis of two related loop-free algorithms for generating integer partitions, Acta Informatica, v16, n2 (1981) pp. 237-252.

[3] T. I. Fenner and G. Loizou, A binary tree representation and related algorithms for generating integer partitions, Comput. J. 23 (1980) pp. 332-337.

[4] T. I. Fenner and G. Loizou, Tree traversal related algorithms for generating integer partitions, SIAM J. Comput. 12 (1983) pp. 551-564.

[5] F. Rubin, Partition of integers, ACM Trans. on Mathematical Software, Vol. 2, No. 4 (1976) pp. 364-374.

[6] E. Horowitz and S. Sahni, Computing partitions with applications to Knapsack, J. of the ACM, Vol. 21, No. 2 (1974) pp. 277-292.

[7] T. V. Narayana, R. M. Mathsen, and J. Sarangi, An algorithm for generating partitions and its applications, J. Combin. Theory, 11 (1971) pp. 54-61.

[8] M. Bader, A study in rule-based logical statistics: the partition of integers, American Laboratory, v32, n24 (2000) pp. 16-19.

[9] R. B. Lin, Efficient data structures for storing the partitions of integers, The 22nd Workshop on Combinatorics and Computation Theory, Tainan, 2005.

[10] K. Ono, Distribution of the partition function modulo m, Annals of Mathematics, 151 (2000), 293-307.